CSE 4165 FINAL EXAM MASTER CHEAT SHEET • SIDE 1: MINIMAL Q1 (JS) & Q2 (PHP)

Time: 1 hr 55 min | 30 Marks | Budget: Q1 30m • Q2 35m • Q3 40m • Zip & Submit 10m

A4 DUPLEX • SIDE 1

Q1 MINIMAL HTML SKELETON (NO EXTRA LABELS/TAGS) — HTML

```
<!-- index.html: short IDs, no extra labels, script at bottom -->
<input type="text" id="a" placeholder="Subject / Meal">
<input type="number" id="b" placeholder="Hours / Calories">
<button onclick="add()">Submit</button>
<div id="res"></div>
<script src="script.js"></script>
```

TYPE 1A: RUNNING TOTAL & STRING ACCUMULATION (NO ARRAY NEEDED!) — TYPE 1A

```
let total = 0, count = 0;
let items = ""; // String accumulator (Just like count++ without array!)
// Or using array: const list = [];

function add() {
  let name = document.getElementById("a").value;
  let val = Number(document.getElementById("b").value);
  if (!name || isNaN(val) || val ≤ 0) return alert("Invalid");

  count++;
  total += val;
  items += name + ", "; // Adds to string just like count++! (Or: list.push(name);)

  let avg = total / count;
  let msg = "";
  if (total ≤ 5)       msg = "Start studying regularly!";
  else if (total ≤ 12) msg = "Good progress!";
  else if (total ≤ 19) msg = "Almost ready!";
  else                 msg = "Weekly goal reached!";

  if (count > 7 && total < 20) msg = "Increase study time / frequent snacking!";

  // Simple line-by-line output:
  document.getElementById("res").innerHTML =
    "Total: " + total + "<br>" +
    "Average: " + avg.toFixed(2) + "<br>" +
    "Subjects: [" + items.slice(0, -2) + "]<br>" + // Array way: list.join(", ")
    "Status: " + msg;

  document.getElementById("a").value = "";
  document.getElementById("b").value = "";
}
```

TYPE 1B: CRITERIA & POINTS SCORER (Password Strength) — TYPE 1B

```
let attempts = 0;
function checkScore() {
  let p = document.getElementById("a").value;
  attempts++;
  let score = 0;
  if (p.length ≥ 6) score += Math.floor((p.length - 6) / 2) * 10;
  if (/[A-Z]/.test(p))      score += 15; // Uppercase
  if (/[a-z]/.test(p))      score += 15; // Lowercase
  if (/[0-9]/.test(p))      score += 20; // Number
  if (/[!@#$%^&*]/.test(p)) score += 25; // Special char

  let lvl = (score ≤ 30) ? "Very Weak" : (score ≤ 50) ? "Weak" :
            (score ≤ 70) ? "Medium" : (score ≤ 90) ? "Strong" : "Very Strong";
  let msg = (score ≥ 100) ? "Perfect Password! Success!" : "Score: " + score + " - " + lvl;
  if (attempts > 8 && score < 71) msg += " | Need practice! Add uppercase, digits & symbols.";
  document.getElementById("res").innerHTML = msg;
}
```

TYPE 1C: MULTI-INPUT SENSOR & FORMULA (Patient Vitals) — TYPE 1C

```
const listA = [], listB = [];
function calc() {
  let vA = Number(document.getElementById("a").value);
  let vB = Number(document.getElementById("b").value);
  if (isNaN(vA) || isNaN(vB)) return;
  listA.push(vA); listB.push(vB);

  let avgA = listA.reduce((s, n) => s + n, 0) / listA.length;
  let avgB = listB.reduce((s, n) => s + n, 0) / listB.length;
  let score = (100 - vB) * 2 + Math.abs(vA - 80) * 0.5;
  let status = (score ≤ 10) ? "SAFE" : (score ≤ 20) ? "WARNING" : "DANGER";

  document.getElementById("res").innerHTML =
    "Avg A: " + avgA.toFixed(1) + " | Avg B: " + avgB.toFixed(1) + " | Score: " + score.toFixed(1) + " | Status: " + status;
}
```

Q2 MINIMAL HTML FORM (JUST NAME ATTRIBUTES) — FORM

```
<!-- Minimal form: only action, method, and name attributes needed -->
<form action="calculate.php" method="POST">
  <input type="number" name="a" placeholder="First Input" step="any"><br><br>
  <input type="number" name="b" placeholder="Second Input" step="any"><br><br>
  <input type="number" name="c" placeholder="Third Input" step="any"><br><br>
  <button type="submit">Calculate</button>
</form>
```

TYPE 2A: CEILING ALLOCATION & WASTE (NO WRAPPER FUNCTIONS!) — TYPE 2A

```
<?php // calculate.php - Type 2A (Direct flat code, saves time!)
$demand   = floatval($_POST['a']);
$rate     = floatval($_POST['b']);
$capacity = floatval($_POST['c']);

$unitsNeeded = ceil($demand / $capacity); // Always round UP with ceil()
$totalCap    = $unitsNeeded * $capacity;
$empty       = $totalCap - $demand;
$wastedCost  = $empty * $rate;

$unusedPct = ($totalCap > 0) ? ($empty / $totalCap) * 100 : 0;
$status = ($unusedPct ≤ 10) ? "Efficient" : (($unusedPct ≤ 25) ? "Acceptable" : "Wasteful");

echo "Units Needed: " . $unitsNeeded . "<br>";
echo "Empty Slots: " . $empty . "<br>";
echo "Wasted Cost: " . $wastedCost . " BDT<br>";
echo "Status: " . $status;
?>
```

TYPE 2B: CREDITS / SALES VS TARGET (DIRECT PROCEDURAL) — TYPE 2B

```
<?php // calculate.php / track.php - Type 2B
$val1   = floatval($_POST['cred']);
$val2   = floatval($_POST['num_course']);
$target = floatval($_POST['tar_cred']);

$total = $val1 * $val2;

// 4-Tier Categorization
$cat = "Poor";
if ($total ≥ 120)     $cat = "Excellent";
elseif ($total ≥ 90)  $cat = "Good";
elseif ($total ≥ 60)  $cat = "Average";

// Target Comparison (Always keep subtraction inside parentheses!)
if ($total == $target)    $state = "Exactly at target (0 diff)";
elseif ($total > $target) $state = "Above target by " . ($total - $target);
else                      $state = "Below target by " . ($target - $total);

echo "Total Credits: " . $total . "<br>";
echo "Progress: " . $cat . "<br>";
echo "Target Comparison: " . $state;
?>
```

TYPE 2C: BEST 2 OF 3 CT MARKS & PASS/FAIL — TYPE 2C

```
<?php // calculate.php - Type 2C
$c1 = floatval($_POST['ct1']);
$c2 = floatval($_POST['ct2']);
$c3 = floatval($_POST['ct3']);
$m  = floatval($_POST['mid']);
$f  = floatval($_POST['final']);

$bestTwoAvg = (($c1 + $c2 + $c3) - min($c1, $c2, $c3)) / 2.0;
$grandTotal = $bestTwoAvg + $m + $f;
$status = ($grandTotal > 54) ? "Passed" : "Failed";

echo "Best 2 CT Avg: " . $bestTwoAvg . "<br>";
echo "Grand Total: " . $grandTotal . "<br>";
echo "Status: <b>" . $status . "</b>";
?>
```

FAST SYNTAX & GOTCHAS REFERENCE

JS Concat: `"A" + b` • **PHP Concat:** `$a . $b`	**PHP Variables:** Always start with `$`
JS Round: `n.toFixed(2)` (returns string!)	**PHP Round:** `ceil($x)`, `number_format($x, 2)`
JS Array: `a.push(x)`, `a.join(", ")`	**PHP Array:** `$a[] = x`, `implode(", ", $a)`
JS Regex: `/[A-Z]/.test(str)`	**PHP Abs:** `abs($a - $b)`, Min: `min($a, $b, $c)`

CSE 4165 FINAL EXAM MASTER CHEAT SHEET • SIDE 2: MINIMAL Q3 (PHP+MYSQL) & 15 SQL ARCHETYPES

Total Marks: 10M (database.sql: 5M + solution.php: 5M) | Direct While Loops (No Bloat) + All 15 SQL Types (242-261)

A4 DUPLEX • SIDE 2

Q3 PART 1: database.sql SKELETON (UNIFIED ITEMS TABLE) — DATABASE.SQL

```
CREATE DATABASE IF NOT EXISTS exam_db;
USE exam_db;
DROP TABLE IF EXISTS items;
CREATE TABLE items (
  id INT PRIMARY KEY,
  name VARCHAR(50),
  category VARCHAR(50),
  price DOUBLE, stock INT, rating DOUBLE, status VARCHAR(50)
);
INSERT INTO items VALUES
(1, 'Laptop',     'Tech',      50000.00, 5,   4.8, 'Active'),
(2, 'Mouse',      'Tech',      500.00,   20,  4.2, 'Active'),
(3, 'Chair',      'Furniture', 3000.00,  8,   4.5, 'Pending'),
(4, 'Desk',       'Furniture', 7200.00,  6,   4.1, 'Active'),
(5, 'Bottle',     'Stationery', 150.00,  40,  3.9, 'Returned'),
(6, 'Cox''s Pen', 'Stationery', 20.00,   100, 4.6, 'Returned');
```

Tips: Escape apostrophe by doubling: `'Cox''s Pen'` • Dates format: `'2024-01-01'`

Q3 PART 2: solution.php (MINIMAL WHILE LOOPS • NO ERROR BLOAT!) — SOLUTION.PHP

```
<?php
$conn = new mysqli("localhost", "root", "", "exam_db");

// Task 1: Category Summary with while loop
echo "<h3>Task 1: Summary</h3>";
$res1 = $conn->query("SELECT category, COUNT(*) AS total, AVG(price) AS
avg_price FROM items GROUP BY category");
while ($row = $res1->fetch_assoc()) {
    echo $row['category'] . " - Total: " . $row['total'] . " - Avg: " .
$row['avg_price'] . "<br>";
}

// Task 2: Conditional UPDATE
$conn->query("UPDATE items SET status = 'Discount' WHERE price < 1000 AN
D status <> 'Pending'");
echo "Task 2 Updated!<br>";

// Task 3: Bonus / Increment with Ceiling Cap
$conn->query("UPDATE items SET price = price + 500 WHERE price > 300 AND
price + 500 ≤ 60000");
echo "Task 3 Updated!<br>";

// Task 4: Subquery / CASE WHEN with while loop
echo "<h3>Task 4: High Value Items</h3>";
$res4 = $conn->query("SELECT name, price FROM items WHERE price > (SELEC
T AVG(price) FROM items)");
while ($row = $res4->fetch_assoc()) {
    echo $row['name'] . ": " . $row['price'] . "<br>";
}

// Net Worth (Single Value)
$res5 = $conn->query("SELECT SUM(price * stock) AS net_worth FROM item
s");
$row5 = $res5->fetch_assoc();
echo "<b>Total Net Worth: " . $row5['net_worth'] . " BDT</b>";

$conn->close();
?>
```

ZERO-CRASH EMERGENCY RULES & GOTCHAS

- **WHERE vs HAVING:** `WHERE` filters rows; `HAVING` filters groups. Never put `COUNT(*)` or `AVG()` in `WHERE`!
- **SQL Order:** `WHERE > GROUP BY > HAVING > ORDER BY`.
- **Execution Order:** Run tasks in exact sequence. Early updates affect later queries.
- **SQL Not Equal:** Use `<>` or `≠`. String in single quotes `'Text'`.
- **XAMPP Run:** Always run via `http://localhost/folder/index.html` (never double click file!).

EXAM WORDING → EXACT SQL MAPPING DICTIONARY

When Question Asks:	Exact SQL Command to Write:
"For each category / department / spot"	`GROUP BY Category` (must be in SELECT too)
"Total number of ... in each ..."	`SELECT Category, COUNT(*) AS Total FROM items GROUP BY Category`
"Only groups with more than N / avg > X"	`HAVING COUNT(*) > N` or `HAVING AVG(Rating) > 4.0`
"Sorted highest to lowest / descending"	`ORDER BY col_name DESC`
"Spot / Item contains word 'Pen' or 'Beach'"	`WHERE ItemName LIKE '%Pen%'`
"Rating between 4.0 and 4.5 (inclusive)"	`WHERE Rating BETWEEN 4.0 AND 4.5`
"If price < 1000 and status != 'Returned' set 'Low'"	`UPDATE items SET Status='Low' WHERE Price < 1000 AND Status <> 'Returned'`
"Add 5000 bonus, only if resulting ≤ 60,000"	`UPDATE items SET Price = Price + 500 WHERE Price > 300 AND (Price + 500) ≤ 60000`
"10% bonus revenue / Increase by 10%"	`UPDATE items SET Price = Price * 1.10 WHERE Condition`
"Apply 10% discount / Reduce by 10%"	`UPDATE items SET Price = Price * 0.90 WHERE Condition`
"Total net worth of entire store"	`SELECT SUM(Price * Stock) AS NetWorth FROM items`
"Label 'Top Seller' if above own category avg"	`CASE WHEN Price > (SELECT AVG(Price) FROM items t2 WHERE t2.Category = items.Category) THEN 'Top Seller' ELSE 'Regular' END`
"Change two columns at once"	`UPDATE items SET Rating = Rating + 0.2, Price = Price * 1.10 WHERE Condition`

ALL 15 SQL QUERY ARCHETYPES (TAGGED WITH PAST PAPERS 242-261) — 15 ARCHETYPES

#	Type & Papers	Exact SQL Query (Runs on items table)
1	Count rows per group 243 251 253 261	`SELECT Category, COUNT(*) AS Total FROM items GROUP BY Category;`
2	Total & avg per group 242 252	`SELECT Category, SUM(Price) AS Revenue, AVG(Price) AS AvgPrice FROM items GROUP BY Category;`
3	Groups with > 1 row 253	`SELECT Status, COUNT(*) AS Total FROM items GROUP BY Status HAVING COUNT(*) > 1;`
4	Top 2 days by date 242	`SELECT SaleDate, SUM(Price) AS Revenue FROM items WHERE SaleDate BETWEEN '2024-01-01' AND '2024-01-31' GROUP BY SaleDate ORDER BY Revenue DESC LIMIT 2;`
5	OR with LIKE substring 261	`SELECT ItemName, Category, Rating FROM items WHERE Rating > 4.5 OR ItemName LIKE '%Pen%' ORDER BY Rating DESC;`
6	Multi-condition filter 261	`SELECT ItemName, Price FROM items WHERE Price > 400 AND Category = 'Electronics';`
7	Whole-table Net Worth 261	`SELECT SUM(Price * Stock) AS NetWorth FROM items;`
8	Count + Avg + Total; HAVING 261	`SELECT Category, COUNT(*) AS Items, AVG(Rating) AS AvgRating, SUM(Stock) AS TotalStock FROM items GROUP BY Category HAVING AVG(Rating) > 4.4 ORDER BY AvgRating DESC;`
9	HAVING calculation 261	`SELECT Category, SUM(Price * Stock) AS Value FROM items GROUP BY Category HAVING SUM(Price * Stock) > 100000 ORDER BY Value DESC;`
10	Distinct dates in HAVING 242	`SELECT Category, AVG(Stock) FROM items GROUP BY Category HAVING COUNT(DISTINCT SaleDate) ≥ 2;`
11	Relative avg (Top Seller) 252	`SELECT i.ItemName, i.Price, CASE WHEN i.Price > (SELECT AVG(i2.Price) FROM items i2 WHERE i2.Category = i.Category) THEN 'Top Seller' ELSE 'Regular Seller' END AS Status FROM items i;`
12	Conditional UPDATE <> 243 251 252 253	`UPDATE items SET Status = 'Low' WHERE Price < 1000 AND Status <> 'Returned';`
13	Bonus with ceiling cap 243 251	`UPDATE items SET Price = Price + 5000 WHERE Price > 3000 AND (Price + 5000) ≤ 60000;`
14	10% discount / raise 252 253 261	`UPDATE items SET Price = Price * 0.90 WHERE Category = 'Furniture'; -- raise: * 1.10`
15	Update 2 columns at once 261	`UPDATE items SET Rating = Rating + 0.2, Price = Price * 1.10 WHERE Rating BETWEEN 4.0 AND 4.5 AND Price > 0;`